SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

CO1-ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code: DSA03	**Course Title: Natural Language Processing**
CO Assessed: CO1 – Imitation (L1)	Assessment: Assessment Tool 1 – Application Based Questions
Weightage: 40% of CIA	CO1 Statement: Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)
Student Name: Rakshitha VB	Reg. No.: 192324004
Section / Batch: Slot B	Date: 10.10.2026

Code of Conduct

I Rakshitha V B 192324004 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: 

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Section / Topic	**Focus**	**Q Nos**
Information Extraction	Regex-Based Information Extraction	1
Pattern Matching	Regex Pattern Matching	2
Regular Expression Patterns	Regex-Based Input Validation	3

Annexure A – Application Based Questions

Solutions

Section 1: Regular Expressions – Resume Information Extraction

Question 1

A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting. Design and implement a Python-based resume information extraction system using Regular Expressions that extracts the name, email addresses, mobile numbers, technical skills (Python, Java, SQL, Machine Learning, NLP) and years of experience, generates a structured summary of the candidate's profile, and displays candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

Approach

Each resume is treated as a plain string. One compiled pattern is written for every field, and each field has its own small function so the extractors can be tested separately. The results are collected into a dictionary that forms the candidate profile, and a separate function checks that profile against the shortlisting rule.

Regular expressions used

Field	Pattern	Reason
Name	^\s*Name\s*[:\-]\s*([A-Za-z][A-Za-z .']+?)\s*$	Picks up the text after the "Name:" label; MULTILINE lets ^ and $ work per line
Email	\b[A-Za-z0-9._%+-]+@[A-Za-z0-9-]+(?:\.[A-Za-z0-9-]+)*\.[A-Za-z]{2,}\b	Local part, @, domain labels and a top-level domain of two or more letters
Mobile	(?<!\d)(?:\+91[\s-]?|0)?([6-9]\d{4}[\s-]?\d{5})(?!\d)	Ten digits starting with 6 to 9, optional +91 or 0 prefix, optional space or hyphen in the middle
Skills	(?<![A-Za-z])Python(?![A-Za-z]) and similar	One pattern per skill, case-insensitive; lookarounds stop "Javascript" from counting as "Java"
Experience	(\d+(?:\.\d+)?)\s*\+?\s*(?:years?|yrs?)\b	A number, optionally decimal, followed by year, years, yr or yrs

Program

```
import re

NAME_PATTERN = re.compile(r"^\s*Name\s*[:\-]\s*([A-Za-z][A-Za-z .']+?)\s*$", re.MULTILINE)
EMAIL_PATTERN = re.compile(r"\b[A-Za-z0-9._%+-]+@[A-Za-z0-9-]+(?:\.[A-Za-z0-9-]+)*\.[A-Za-
z]{2,}\b")
MOBILE_PATTERN = re.compile(r"(?<!\d)(?:\+91[\s-]?|0)?([6-9]\d{4}[\s-]?\d{5})(?!\d)")
EXPERIENCE_PATTERN = re.compile(r"(\d+(?:\.\d+)?)\s*\+?\s*(?:years?|yrs?)\b", re.IGNORECASE)
SKILL_LIST = ["Python", "Java", "SQL", "Machine Learning", "NLP"]
SKILL_PATTERNS = {
    skill: re.compile(r"(?<![A-Za-z])" + re.escape(skill).replace(r"\ ", r"\s+") + r"(?![A-Za-
z])", re.IGNORECASE)
    for skill in SKILL_LIST
}



def extract_name(text):
    match = NAME_PATTERN.search(text)
    return match.group(1).strip() if match else "Not found"



def extract_emails(text):
    return EMAIL_PATTERN.findall(text)
```

```
def extract_mobiles(text):
    return [re.sub(r"[\s-]", "", number) for number in MOBILE_PATTERN.findall(text)]


def extract_skills(text):
    return [skill for skill, pattern in SKILL_PATTERNS.items() if pattern.search(text)]


def extract_experience(text):
    values = [float(value) for value in EXPERIENCE_PATTERN.findall(text)]
    return max(values) if values else 0.0


def build_profile(text):
    return {
        "name": extract_name(text),
        "emails": extract_emails(text),
        "mobiles": extract_mobiles(text),
        "skills": extract_skills(text),
        "experience": extract_experience(text),
    }


def is_eligible(profile, min_experience=2.0, required_skill="Python"):
    return profile["experience"] >= min_experience and required_skill in profile["skills"]


def print_profile(index, profile):
    print(f"Candidate {index}")
    print("-" * 52)
    print(f"Name        : {profile['name']}")
    print(f"Email       : {', '.join(profile['emails']) or 'Not found'}")
    print(f"Mobile      : {', '.join(profile['mobiles']) or 'Not found'}")
    print(f"Skills      : {', '.join(profile['skills']) or 'None detected'}")
    unit = "year" if profile["experience"] == 1 else "years"
    print(f"Experience  : {profile['experience']:g} {unit}")
    print(f"Eligibility : {'Eligible' if is_eligible(profile) else 'Not eligible'}")
    print()


def main():
    resumes = [
        """Name: Arun Kumar
Email: arun.kumar@gmail.com
Mobile: +91 98765 43210
Skills: Python, SQL, Machine Learning
Experience: 3 years as a data analyst""",
        """Name: Priya Sharma
Email: priya_sharma92@outlook.com
Phone: 9123456780
Skills: Java, SQL
Experience: 4.5 years in backend development""",
        """Name: Rahul Menon
Email: rahul.menon@company.co.in
Contact: 8012345678
Skills: python, NLP, machine learning
Experience: 1 year of internship experience""",
        """Name: Divya Lakshmi
Email: divya.lakshmi@yahoo.com
Mobile: +91-70123-45678
Skills: Python, Java, NLP
Experience: 2 years in text analytics""",
        """Name: Karthik Raja
Email: karthik.raja@@mail
Mobile: 12345
Skills: Excel, Tableau
Experience: 6 years in reporting""",
    ]

    profiles = [build_profile(resume) for resume in resumes]

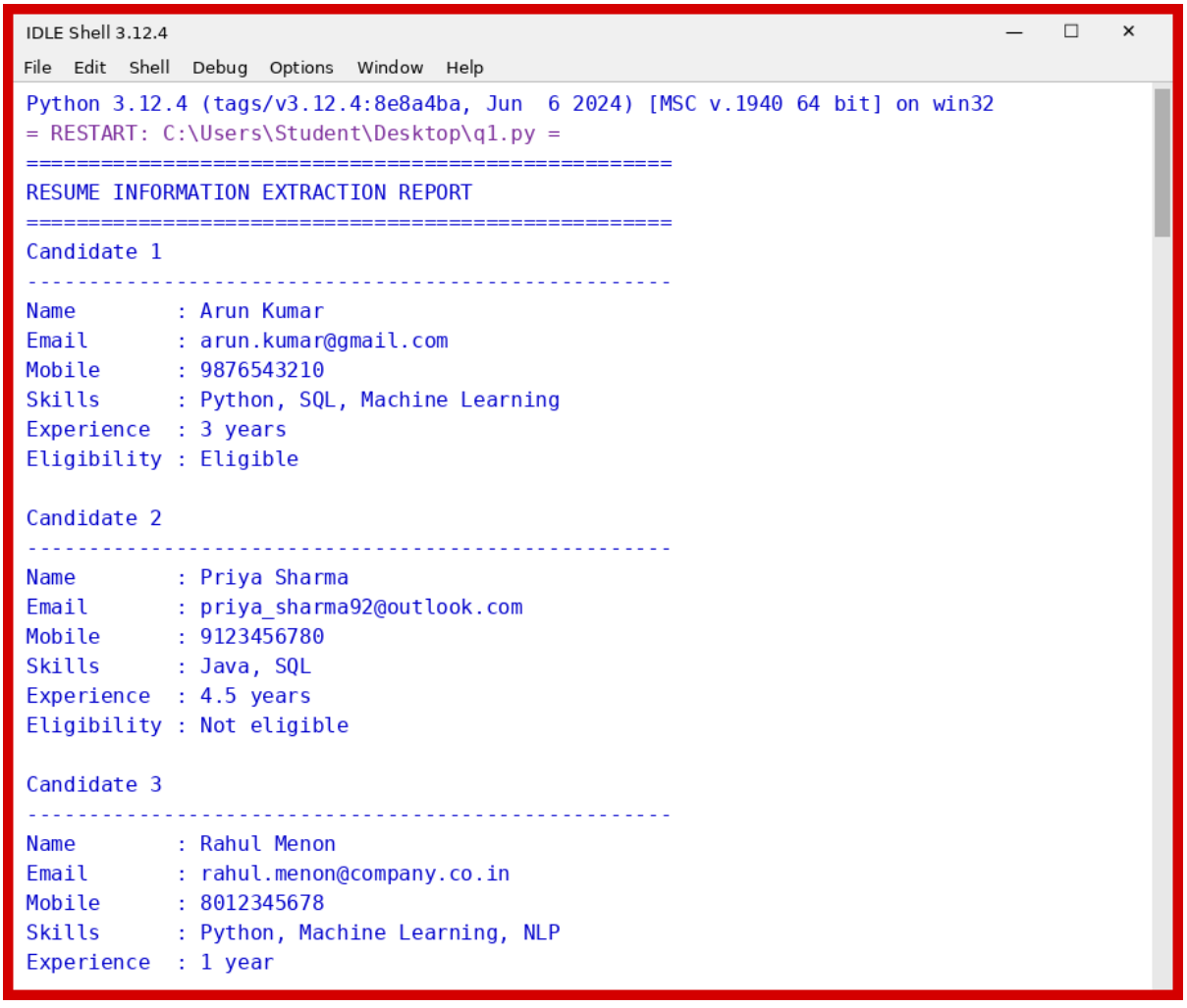
    print("=" * 52)
    print("RESUME INFORMATION EXTRACTION REPORT")
    print("=" * 52)
    for index, profile in enumerate(profiles, start=1):
        print_profile(index, profile)

    shortlisted = [profile for profile in profiles if is_eligible(profile)]
```

```
    print("=" * 52)
    print("SHORTLISTED CANDIDATES (>= 2 years and Python)")
    print("=" * 52)
    if shortlisted:
        for profile in shortlisted:
            print(f"{profile['name']:<16} | {profile['experience']:g} yrs | {',
'.join(profile['skills'])}")
    else:
        print("No candidate satisfies the criteria.")
    print(f"\nTotal resumes processed : {len(profiles)}")
    print(f"Total shortlisted       : {len(shortlisted)}")


if __name__ == "__main__":
    main()
```

Output



```
IDLE Shell 3.12.4
File  Edit  Shell  Debug  Options  Window  Help
Eligibility : Not eligible

Candidate 4
--------------------------------------------------
Name        : Divya Lakshmi
Email       : divya.lakshmi@yahoo.com
Mobile      : 7012345678
Skills      : Python, Java, NLP
Experience  : 2 years
Eligibility : Eligible

Candidate 5
--------------------------------------------------
Name        : Karthik Raja
Email       : Not found
Mobile      : Not found
Skills      : None detected
Experience  : 6 years
Eligibility : Not eligible

==================================================
SHORTLISTED CANDIDATES (>= 2 years and Python)
==================================================
Arun Kumar       | 3 yrs | Python, SQL, Machine Learning
Divya Lakshmi    | 2 yrs | Python, Java, NLP

Total resumes processed : 5
Total shortlisted       : 2
>>>
```

Graphical User Interface

The same functions are wrapped in a Tkinter window (gui_q1.py). Resume text is pasted or loaded in the left panel, the extracted details appear in the table with eligible candidates shaded green, and the shortlist is shown separately.

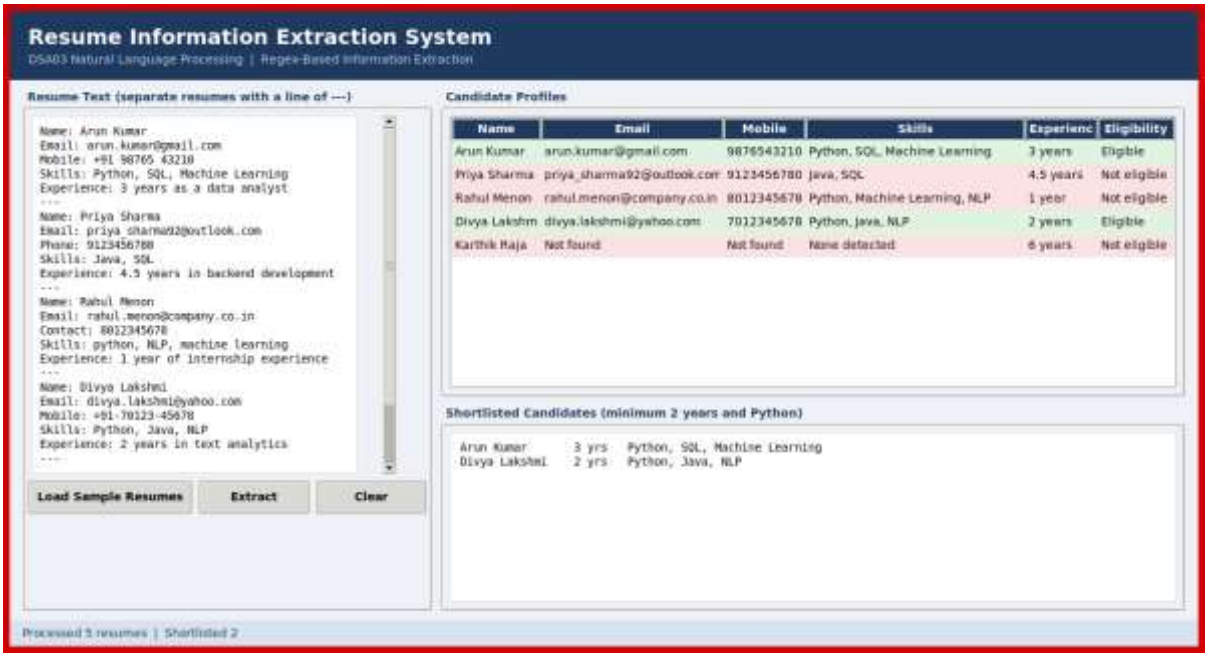


Figure 1: Resume extraction interface after processing the sample resumes

Justification of the output

- Arun Kumar: the +91 prefix and the space inside the number are removed, so the mobile number is stored as 9876543210. He has three years of experience and knows Python, so he is shortlisted.
- Priya Sharma: 4.5 years is read correctly as a decimal value. Her skills are Java and SQL only, so she fails the Python condition even though her experience is above the minimum.
- Rahul Menon: "python" in lower case is still detected because the search ignores case. He has only 1 year of experience, so he is not shortlisted.
- Divya Lakshmi: exactly 2 years with Python. The check uses >=, so she is included. This is the boundary case.
- Karthik Raja: the email "karthik.raja@@mail" has a double @ and no valid domain, and "12345" is too short to be a mobile number, so both fields are reported as not found. None of the five skills appear in his resume, so the skills field is empty even though he has 6 years of experience.

Two of the five candidates, Arun Kumar and Divya Lakshmi, satisfy both conditions, which matches the shortlist printed by the program.

Section 2: Regular Expressions – Product Search System

Question 2

An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching. Design and implement a Python-based product search system using Regular Expressions that searches products using an exact keyword, performs prefix-based, suffix-based and partial keyword searches, performs case-insensitive searches, displays all matching product names, and generates a report showing the total number of matching products for each search.

Approach

The only difference between the four search types is how the keyword is wrapped before it is compiled. The keyword is first passed through re.escape so that characters such as "+" or "." typed by a user are treated as ordinary text. Case-insensitive searching is controlled by the re.IGNORECASE flag, which is switched on by default and can be switched off to show the difference. Each search returns a list of products, and a final function prints a table with the count for every search.

Regular expressions used

Search type	Pattern	Meaning
Exact	\bkeyword\b	The keyword must appear as a complete word
Prefix	\bkeyword\w*	A word that begins with the keyword
Suffix	\w*keyword\b	A word that ends with the keyword
Partial	keyword	The keyword may appear anywhere, even inside a word
Case-insensitive	re.IGNORECASE	Flag applied while compiling; upper and lower case are treated alike

Program

```
import re

PRODUCTS = [
    "Apple iPhone 15",
    "Samsung Galaxy Phone",
    "Wireless Bluetooth Headphones",
    "Noise Cancelling Headphone",
    "Smart LED Television",
    "Phone Case Silicone",
    "Laptop Backpack",
    "Gaming Laptop",
    "Laptop Stand Adjustable",
    "Mechanical Keyboard",
    "Wireless Keyboard and Mouse",
    "Smartwatch Series 8",
    "Portable Power Bank",
    "USB Phone Charger",
    "Headphone Stand",
]


def build_pattern(keyword, mode):
    if not keyword or not keyword.strip():
        raise ValueError("Search keyword cannot be empty.")
    word = re.escape(keyword.strip())
    patterns = {
        "exact": rf"\b{word}\b",
        "prefix": rf"\b{word}\w*",
        "suffix": rf"\w*{word}\b",
        "partial": rf"{word}",
    }
    if mode not in patterns:
        raise ValueError(f"Unknown search mode: {mode}")
    return patterns[mode]


def search_products(products, keyword, mode, ignore_case=True):
    flags = re.IGNORECASE if ignore_case else 0
```

```
    regex = re.compile(build_pattern(keyword, mode), flags)
    return [product for product in products if regex.search(product)]


def run_search(products, keyword, mode, ignore_case=True):
    try:
        matches = search_products(products, keyword, mode, ignore_case)
    except ValueError as error:
        print(f"Error: {error}\n")
        return None
    case_label = "Case-insensitive" if ignore_case else "Case-sensitive"
    print(f"Search type : {mode.capitalize()} ({case_label})")
    print(f"Keyword     : '{keyword}'")
    if matches:
        for position, name in enumerate(matches, start=1):
            print(f"  {position}. {name}")
    else:
        print("  No matching products found.")
    print(f"Matches     : {len(matches)}\n")
    return len(matches)


def print_report(results):
    print("=" * 62)
    print("SEARCH SUMMARY REPORT")
    print("=" * 62)
    print(f"{'No.':<5}{'Search Type':<22}{'Keyword':<16}{'Matches':>8}")
    print("-" * 62)
    for number, (label, keyword, count) in enumerate(results, start=1):
        print(f"{number:<5}{label:<22}{keyword:<16}{count:>8}")
    print("-" * 62)
    print(f"{'Total searches performed':<43}{len(results):>19}")
    print(f"{'Total matches across searches':<43}{sum(r[2] for r in results):>19}")


def main():
    searches = [
        ("Exact", "laptop", "exact", True),
        ("Prefix", "head", "prefix", True),
        ("Suffix", "phone", "suffix", True),
        ("Partial", "board", "partial", True),
        ("Case-insensitive", "WIRELESS", "exact", True),
        ("Case-sensitive", "WIRELESS", "exact", False),
        ("Exact", "tablet", "exact", True),
    ]

    print("=" * 62)
    print("PRODUCT SEARCH SYSTEM")
    print("=" * 62)
    print(f"Catalogue size: {len(PRODUCTS)} products\n")

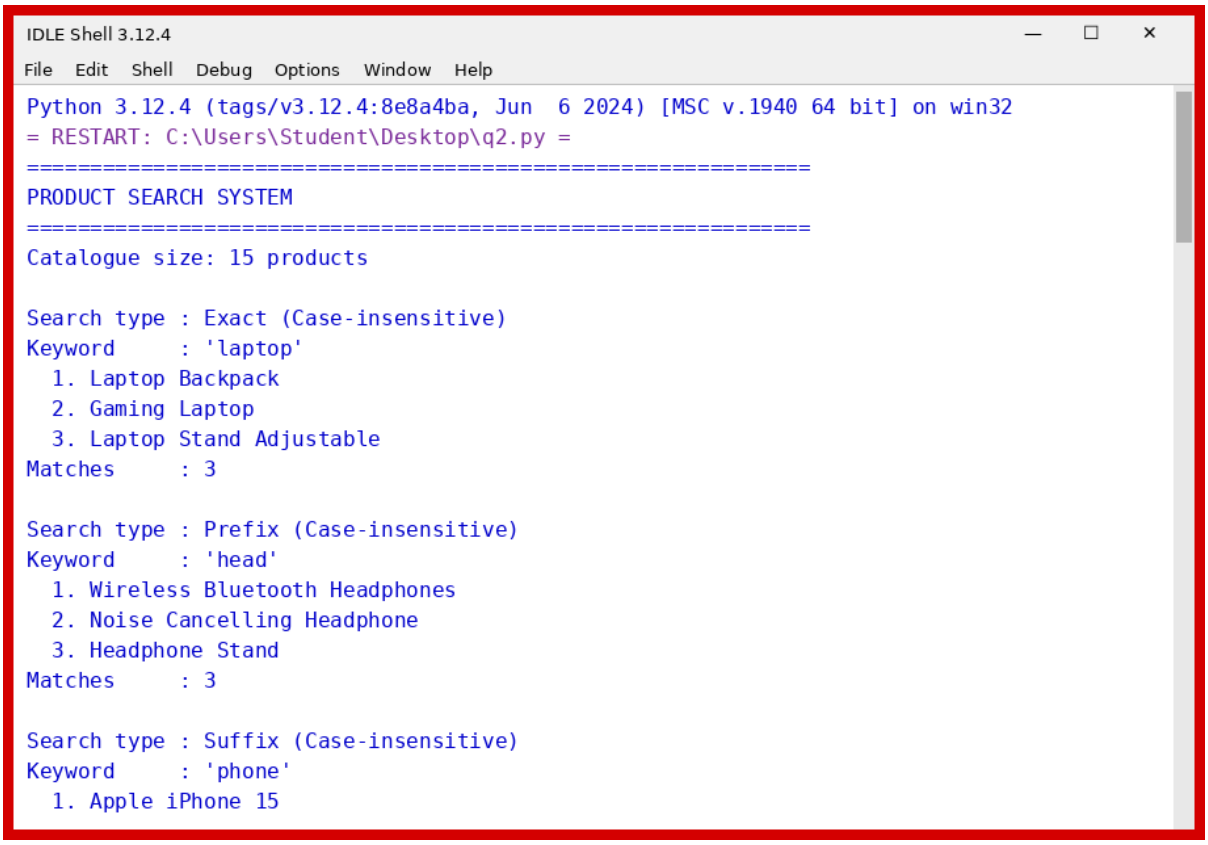
    results = []
    for label, keyword, mode, ignore_case in searches:
        count = run_search(PRODUCTS, keyword, mode, ignore_case)
        results.append((label, keyword, count))

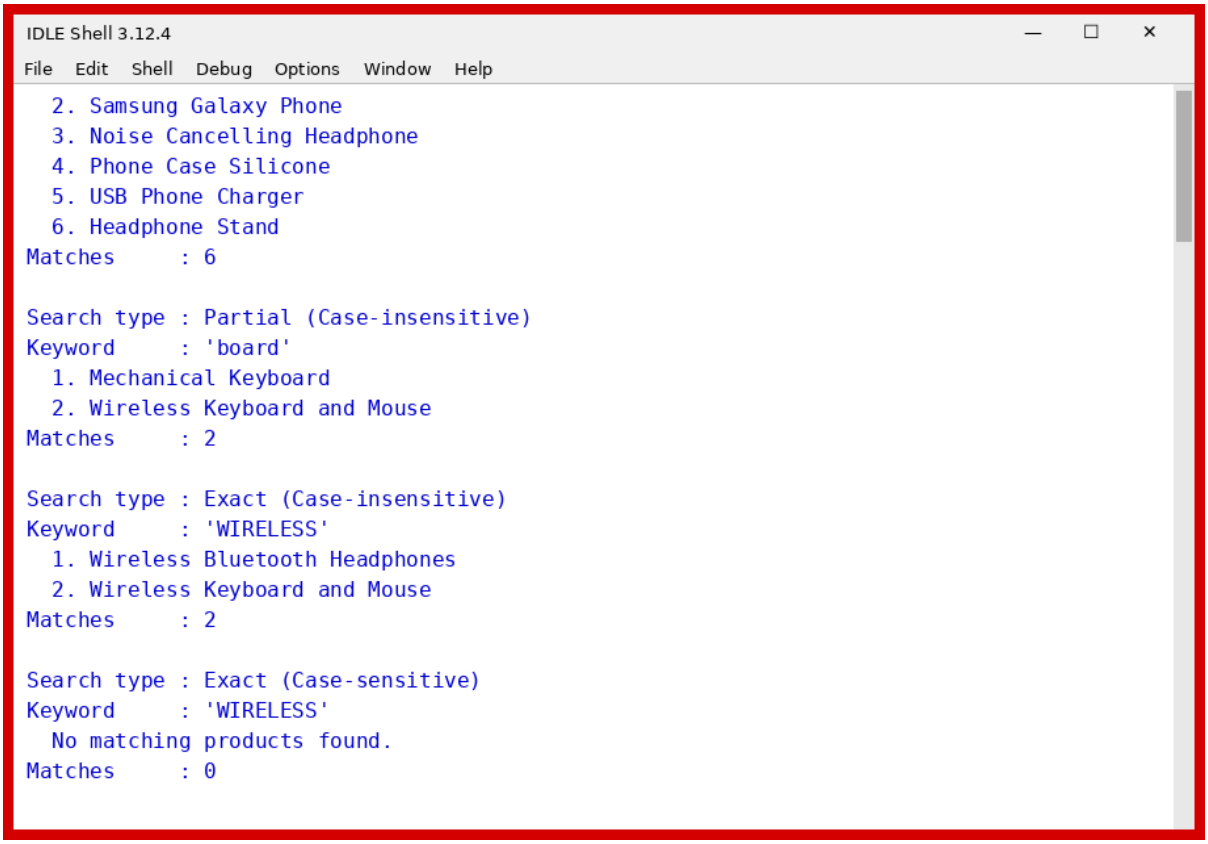
    run_search(PRODUCTS, "   ", "exact")

    print_report(results)


if __name__ == "__main__":
    main()
```

Output


IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024) [MSC v.1940 64 bit] on win32
= RESTART: C:\Users\Student\Desktop\q2.py =
PRODUCT SEARCH SYSTEM
Catalogue size: 15 products
Search type : Exact (Case-insensitive)
Keyword : 'laptop'
1. Laptop Backpack
2. Gaming Laptop
3. Laptop Stand Adjustable
Matches : 3
Search type : Prefix (Case-insensitive)
Keyword : 'head'
1. Wireless Bluetooth Headphones
2. Noise Cancelling Headphone
3. Headphone Stand
Matches : 3
Search type : Suffix (Case-insensitive)
Keyword : 'phone'
1. Apple iPhone 15


IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
2. Samsung Galaxy Phone
3. Noise Cancelling Headphone
4. Phone Case Silicone
5. USB Phone Charger
6. Headphone Stand
Matches : 6
Search type : Partial (Case-insensitive)
Keyword : 'board'
1. Mechanical Keyboard
2. Wireless Keyboard and Mouse
Matches : 2
Search type : Exact (Case-insensitive)
Keyword : 'WIRELESS'
1. Wireless Bluetooth Headphones
2. Wireless Keyboard and Mouse
Matches : 2
Search type : Exact (Case-sensitive)
Keyword : 'WIRELESS'
No matching products found.
Matches : 0

```
IDLE Shell 3.12.4
File  Edit  Shell  Debug  Options  Window  Help
Search type : Exact (Case-insensitive)
Keyword     : 'tablet'
  No matching products found.
Matches     : 0

Error: Search keyword cannot be empty.

============================================================
SEARCH SUMMARY REPORT
============================================================
No.  Search Type          Keyword         Matches
------------------------------------------------------------
1    Exact                laptop                3
2    Prefix               head                  3
3    Suffix               phone                 6
4    Partial              board                 2
5    Case-insensitive     WIRELESS              2
6    Case-sensitive       WIRELESS              0
7    Exact                tablet                0
------------------------------------------------------------
Total searches performed                         7
Total matches across searches                   16
>>>
```

Graphical User Interface

The search logic is placed in a separate Tkinter window (gui_q2.py). The keyword, search type and case option are chosen at the top, the catalogue highlights every matching product, and the report table adds one row per search with the running totals.

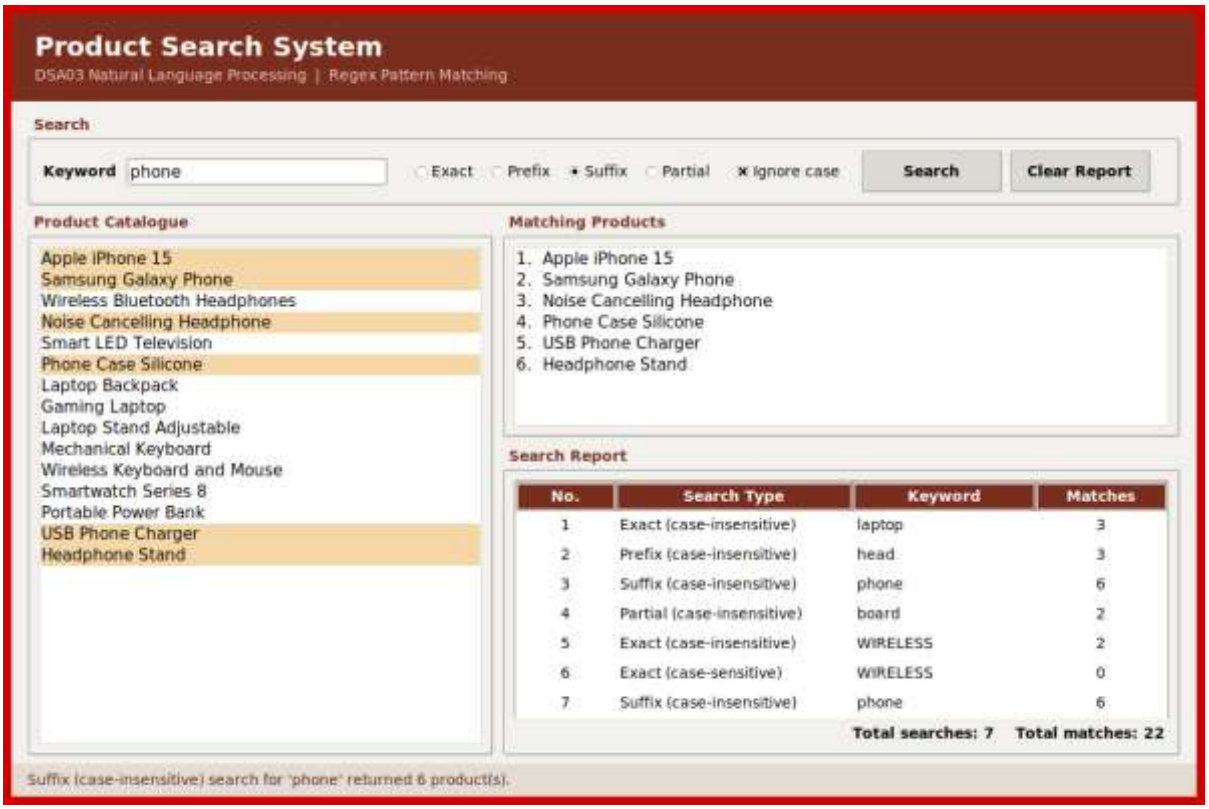


Figure 2: Product search interface with the search report

Justification of the output

- Exact "laptop" returns three products. "Laptop Backpack", "Gaming Laptop" and "Laptop Stand Adjustable" contain the word on its own.
- Prefix "head" returns "Headphones", "Headphone" and "Headphone Stand", all of which begin with the keyword.
- Suffix "phone" returns six products. "Apple iPhone 15" is included because "iPhone" ends in "phone", and "Phone Case Silicone" is included because the word itself is "Phone". "Smartwatch Series 8" and similar items are correctly left out.
- Partial "board" returns the two keyboards, since "board" sits inside the word "Keyboard" and is neither a prefix nor a whole word there.
- Searching for "WIRELESS" with case ignored finds two products, while the same search with case-sensitivity returns zero. This confirms the flag works.
- "tablet" returns zero matches and the program prints a clear message instead of an empty list. A blank keyword raises a ValueError that is caught and reported, so the program does not crash.

The summary report adds up to 16 matches across 7 searches, which agrees with the individual counts shown above it.

Section 3: Regular Expressions – University Registration System

Question 3

A university is developing an automated online registration portal to verify student information before course enrollment. Design and implement a Python-based validation system using Regular Expressions that validates the student register number, institutional email address, course code, semester information and mobile number, displays appropriate validation messages for each field, and generates a final registration status report indicating whether the registration is successful.

Assumed formats

The question does not give the exact formats, so the following rules were assumed. They are kept in one dictionary in the program, which makes it easy to change them to the real university rules.

Field	Pattern	Rule
Register Number	^(?:19|2[0-6])\d{7}$	Nine digits; the first two digits are the admission year (19 to 26)
Institutional Email	^[A-Za-z][A-Za-z0-9._-]*@simats\.edu\.in$	Must start with a letter and end with the @simats.edu.in domain
Course Code	^[A-Z]{3}\d{2}$	Three capital letters followed by two digits, for example DSA03
Semester	^(?:Semester\s)?[1-8]$	A number from 1 to 8, with an optional "Semester " prefix
Mobile Number	^(?:\+91[\s-]?)?[6-9]\d{9}$	Ten digits starting with 6 to 9, optional +91 prefix

Approach

Each field is checked with fullmatch, so the whole value must follow the pattern and not just a part of it. validate_field returns a pair of a pass flag and a message, which keeps the checking separate from the printing. validate_student runs all five fields, and registration_report prints the messages and declares the registration successful only when every field passes.

Program

```
import re

RULES = {
    "Register Number": (
        re.compile(r"^(?:19|2[0-6])\d{7}$"),
        "9 digits starting with an admission year between 19 and 26",
    ),
    "Institutional Email": (
        re.compile(r"^[A-Za-z][A-Za-z0-9._-]*@simats\.edu\.in$"),
        "must be of the form name@simats.edu.in",
    ),
    "Course Code": (
        re.compile(r"^[A-Z]{3}\d{2}$"),
        "three capital letters followed by two digits, e.g. DSA03",
    ),
    "Semester": (
        re.compile(r"^(?:Semester\s)?[1-8]$"),
        "a number from 1 to 8, optionally prefixed with 'Semester '",
    ),
    "Mobile Number": (
        re.compile(r"^(?:\+91[\s-]?)?[6-9]\d{9}$"),
        "10 digits starting with 6-9, optional +91 prefix",
    ),
}


def validate_field(field, value):
    pattern, requirement = RULES[field]
    if not isinstance(value, str) or not value.strip():
        return False, f"{field}: Invalid - field is empty"
    if pattern.fullmatch(value.strip()):
        return True, f"{field}: Valid"
```

```
    return False, f"{field}: Invalid - {requirement}"


def validate_student(record):
    results = []
    for field in RULES:
        results.append(validate_field(field, record.get(field, "")))
    return results


def registration_report(record):
    results = validate_student(record)
    print(f"Student record : {record.get('Register Number', '') or '(blank)'}")
    print("-" * 66)
    for _, message in results:
        print(message)
    failed = sum(1 for passed, _ in results if not passed)
    print("-" * 66)
    if failed == 0:
        print("REGISTRATION STATUS: SUCCESSFUL")
    else:
        print(f"REGISTRATION STATUS: FAILED ({failed} field(s) invalid)")
    print()
    return failed == 0


def main():
    records = [
        {
            "Register Number": "192311025",
            "Institutional Email": "arun.kumar@simats.edu.in",
            "Course Code": "DSA03",
            "Semester": "5",
            "Mobile Number": "+91 9876543210",
        },
        {
            "Register Number": "19231102",
            "Institutional Email": "priya.s@gmail.com",
            "Course Code": "dsa03",
            "Semester": "9",
            "Mobile Number": "5123456789",
        },
        {
            "Register Number": "212411067",
            "Institutional Email": "rahul_m@simats.edu.in",
            "Course Code": "CSA12",
            "Semester": "Semester 3",
            "Mobile Number": "98765432",
        },
        {
            "Register Number": "202211340",
            "Institutional Email": "divya.l@simats.edu.in",
            "Course Code": "ITA101",
            "Semester": "8",
            "Mobile Number": "7012345678",
        },
        {
            "Register Number": "",
            "Institutional Email": "",
            "Course Code": "",
            "Semester": "",
            "Mobile Number": "",
        },
    ]

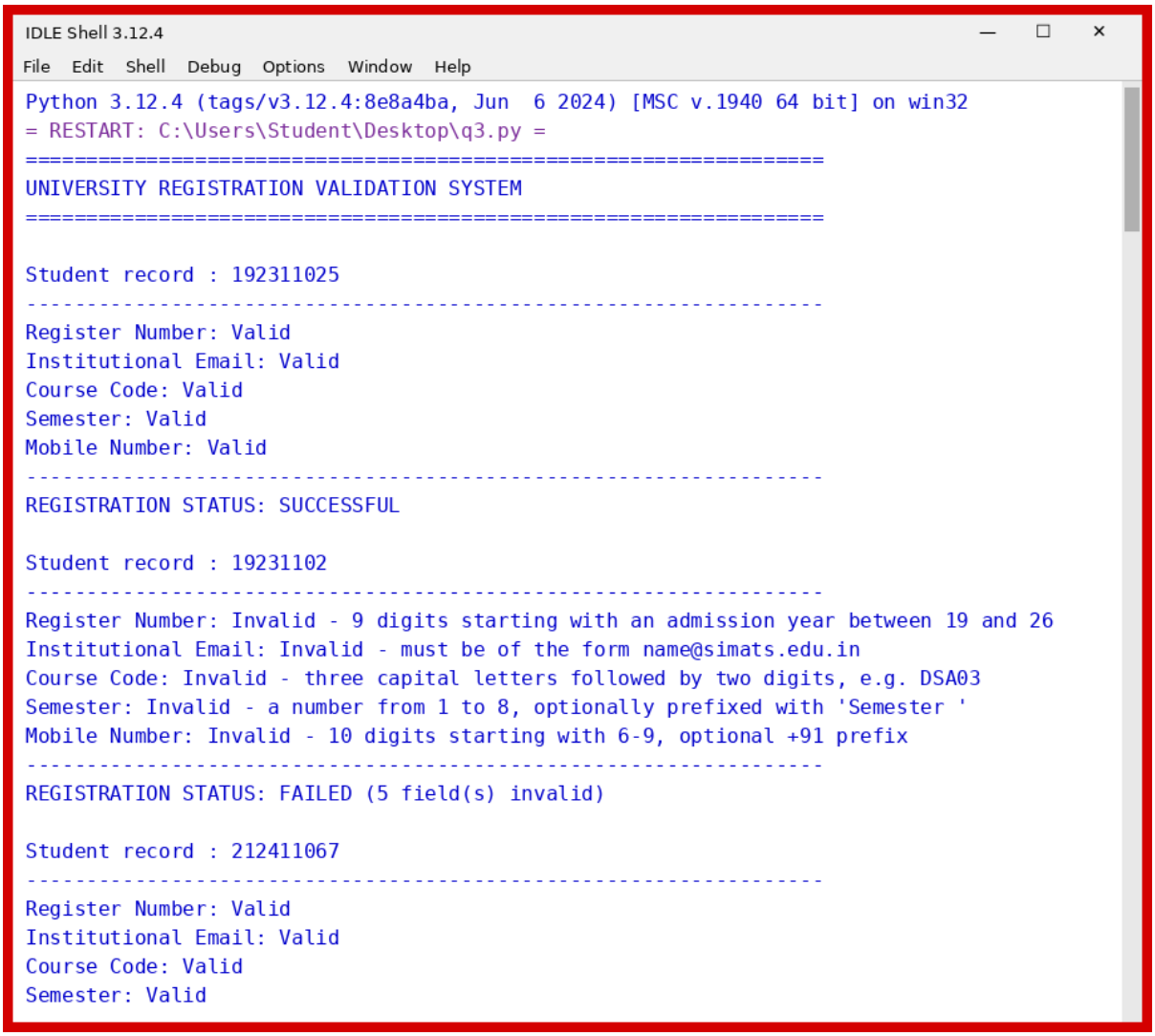
    print("=" * 66)
    print("UNIVERSITY REGISTRATION VALIDATION SYSTEM")
    print("=" * 66)
    print()

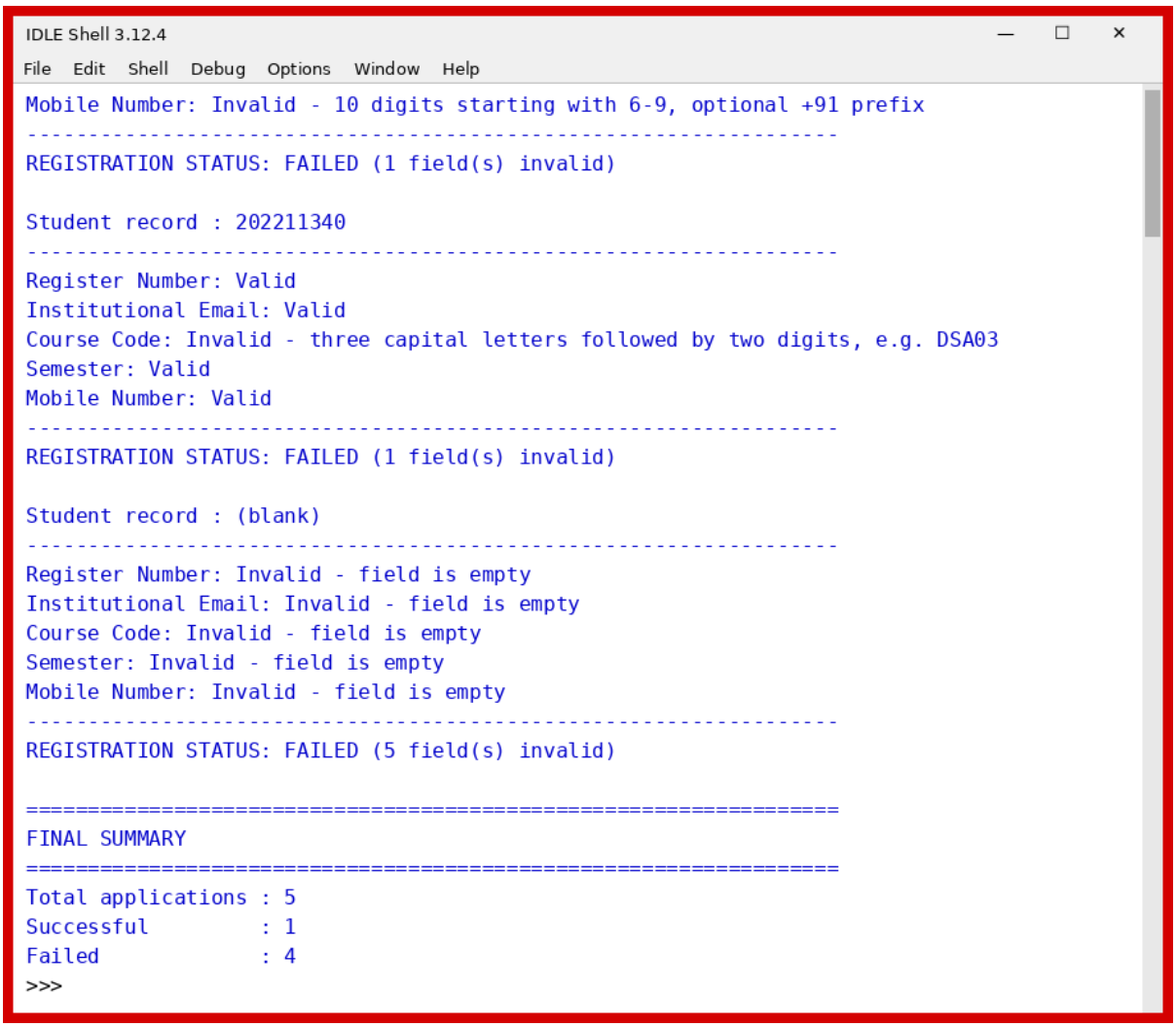
    outcomes = [registration_report(record) for record in records]

    print("=" * 66)
    print("FINAL SUMMARY")
    print("=" * 66)
    print(f"Total applications : {len(outcomes)}")
    print(f"Successful         : {sum(outcomes)}")
    print(f"Failed             : {len(outcomes) - sum(outcomes)}")


if __name__ == "__main__":
    main()
```

Output


IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun 6 2024) [MSC v.1940 64 bit] on win32
= RESTART: C:\Users\Student\Desktop\q3.py =
UNIVERSITY REGISTRATION VALIDATION SYSTEM
Student record : 192311025
Register Number: Valid
Institutional Email: Valid
Course Code: Valid
Semester: Valid
Mobile Number: Valid
REGISTRATION STATUS: SUCCESSFUL
Student record : 19231102
Register Number: Invalid - 9 digits starting with an admission year between 19 and 26
Institutional Email: Invalid - must be of the form name@simats.edu.in
Course Code: Invalid - three capital letters followed by two digits, e.g. DSA03
Semester: Invalid - a number from 1 to 8, optionally prefixed with 'Semester '
Mobile Number: Invalid - 10 digits starting with 6-9, optional +91 prefix
REGISTRATION STATUS: FAILED (5 field(s) invalid)
Student record : 212411067
Register Number: Valid
Institutional Email: Valid
Course Code: Valid
Semester: Valid



Graphical User Interface

The validation rules are used in a Tkinter form (gui_q3.py). Each field shows a tick or a cross with its message, and the banner at the bottom gives the final registration status. Both a failed and a successful registration are shown.

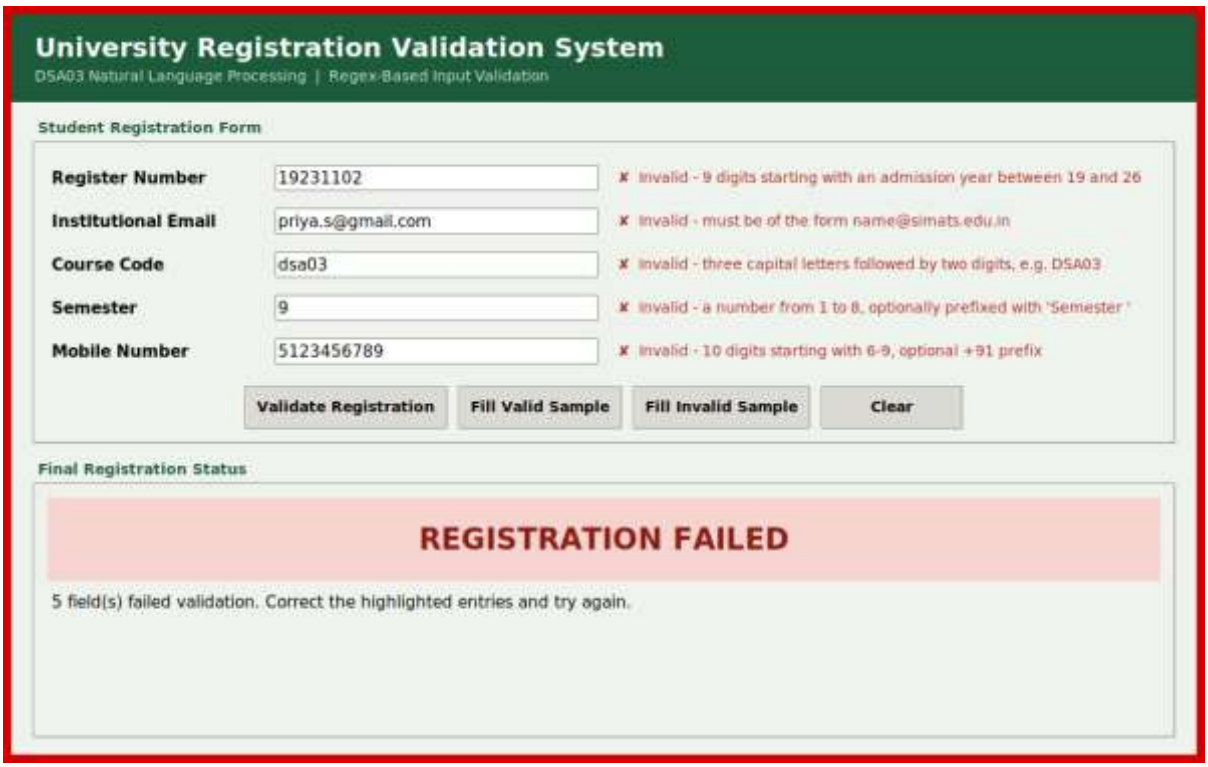


Figure 3: Registration form with invalid entries

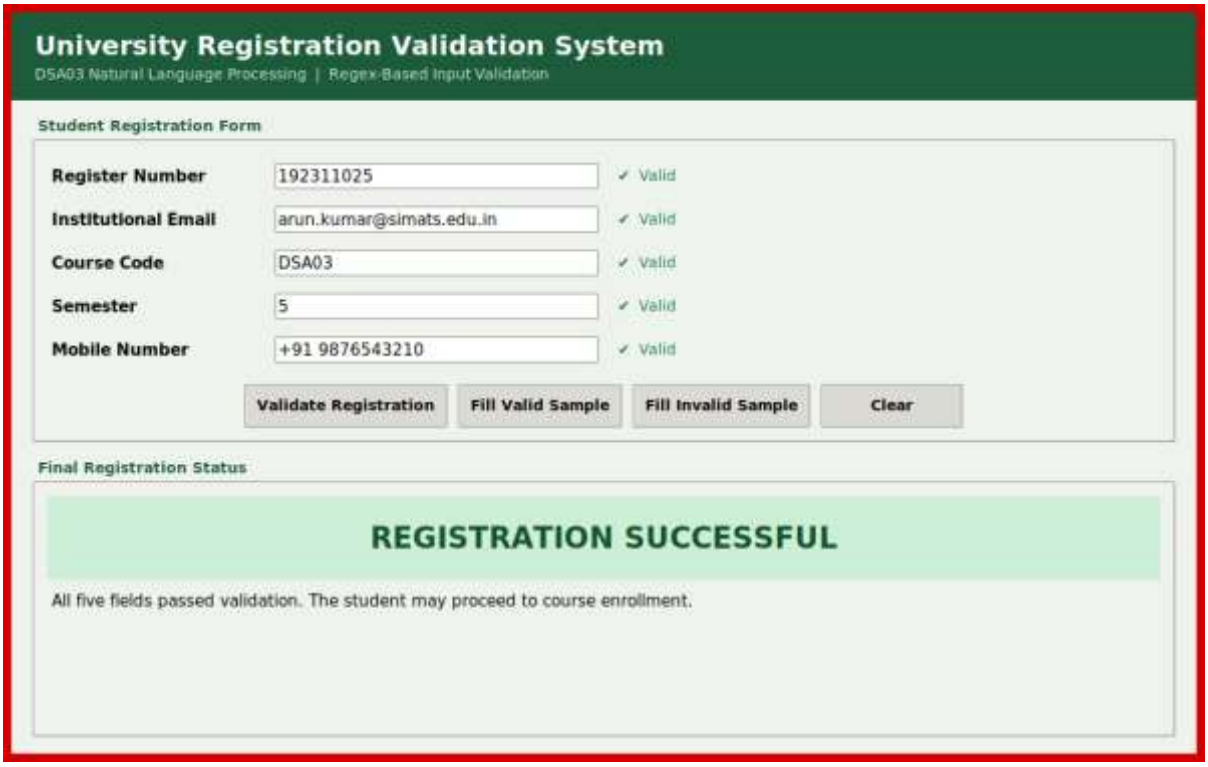


Figure 4: Registration form with valid entries

Justification of the output

- Record 1 meets every rule, including the "+91 " prefix with a space in the mobile number, so the registration is successful.
- Record 2 is invalid in all five fields: the register number has only 8 digits, the email uses gmail.com, the course code is in lower case, semester 9 is outside the range 1 to 8, and the mobile number starts with 5.

- Record 3 passes four fields, including "Semester 3" and the underscore in the email. The mobile number has only 8 digits, so the registration fails on that one field.
- Record 4 fails only the course code, because "ITA101" has three digits instead of two.
- Record 5 has blank values. These are handled before the regex runs, so each field reports that it is empty and the program does not raise an error.

One of the five applications is successful and four fail, and the final summary reports the same numbers.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q. No. / Criteria	Understanding of Problem Context	Application of Concepts	Problem-Solving Approach	Accuracy of Solution	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____